EXPERIMENT 6

Implement a bidirectional recurrent neural network (RNN) to predict sequences in timeseries data. Train the RNN and compare its performance with a traditional feed-forward neural network for sequence-based tasks.

```
import numpy as np
import pandas as pd
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, SimpleRNN, Bidirectional
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_squared_error

# Load Airline Passenger dataset
url = "https://raw.githubusercontent.com/jbrownlee/Datasets/master/airline-passengers.csv"

data = pd.read_csv(url)

values = data["Passengers"].values.reshape(-1, 1)

# Normalize data
scaler = MinMaxScaler()
values = scaler.fit_transform(values)

# Create sequences
X = []
y = []

for i in range(12, len(values)):
    X.append(values[i-12:i])
    y.append(values[i])

X = np.array(X)
y = np.array(y)

# Train-test split
split = int(len(X) * 0.8)

X_train = X[:split]
X_test = X[split:]

y_train = y[:split]
y_test = y[split:]

# --------------------------------
# Bidirectional RNN
# --------------------------------

bi_model = Sequential([
    Bidirectional(
        SimpleRNN(32),
        input_shape=(12, 1)
    ),
    Dense(16, activation="relu"),
    Dense(1)
])

bi_model.compile(
    optimizer="adam",
    loss="mse"
)

print("Training Bidirectional RNN")

bi_model.fit(
    X_train,
    y_train,
    epochs=20,
    batch_size=16,
    verbose=1
)

bi_pred = bi_model.predict(X_test)

bi_rmse = np.sqrt(
    mean_squared_error(y_test, bi_pred)
)
```

```
print("Bidirectional RNN RMSE:", bi_rmse)


# --------------------------------
# Feed-Forward Neural Network
# --------------------------------

ff_model = Sequential([
    Dense(32, activation="relu", input_shape=(12,)),
    Dense(16, activation="relu"),
    Dense(1)
])

ff_model.compile(
    optimizer="adam",
    loss="mse"
)

print("\nTraining Feed-Forward Neural Network")

ff_model.fit(
    X_train.reshape(X_train.shape[0], 12),
    y_train,
    epochs=20,
    batch_size=16,
    verbose=1
)

ff_pred = ff_model.predict(
    X_test.reshape(X_test.shape[0], 12)
)

ff_rmse = np.sqrt(
    mean_squared_error(y_test, ff_pred)
)

print("Feed-Forward NN RMSE:", ff_rmse)


# --------------------------------
# Comparison
# --------------------------------

print("\n----- PERFORMANCE COMPARISON -----")
print("Bidirectional RNN RMSE :", bi_rmse)
print("Feed-Forward NN RMSE   :", ff_rmse)

if bi_rmse < ff_rmse:
    print("Bidirectional RNN performs better.")
else:
    print("Feed-Forward NN performs better.")
```

```
/usr/local/lib/python3.13/dist-packages/keras/src/layers/rnn/bidirectional.py:107: UserWarning: Do not pass an `input_shap
  super().__init__(**kwargs)
Training Bidirectional RNN
Epoch 1/20
7/7 ━━━━━━━━━━━━━━━━━━━━ 4s 5ms/step - loss: 0.0357
Epoch 2/20
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0101
Epoch 3/20
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0066
Epoch 4/20
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0053
Epoch 5/20
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0029
Epoch 6/20
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - loss: 0.0025
Epoch 7/20
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0018
Epoch 8/20
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0016
Epoch 9/20
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0012
Epoch 10/20
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 9.8754e-04
Epoch 11/20
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0010
Epoch 12/20
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 9.1990e-04
Epoch 13/20
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0010
Epoch 14/20
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 8.0599e-04
Epoch 15/20
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 6.7300e-04
```

```
Epoch 16/20
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 7.3131e-04
Epoch 17/20
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - loss: 6.9494e-04
Epoch 18/20
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - loss: 6.7262e-04
Epoch 19/20
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 6.5531e-04
Epoch 20/20
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 6.3424e-04
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 165ms/step
Bidirectional RNN RMSE: 0.06659100168695606

Training Feed-Forward Neural Network
Epoch 1/20
/usr/local/lib/python3.13/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape`/`inp
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
7/7 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1297
Epoch 2/20
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.0559
Epoch 3/20
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 0.0215
Epoch 4/20
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 0.0074
```